

Assignment - 22

Runtime Polymorphism:

Ability of a function to be dynamically bound at runtime based on object being referenced.

- Achieved using function overriding (virtual C)

Diff betⁿ
overloading

multiple () with same name
but diff parameter ...

Called in Same Class

(1) Not Required

(2) Compile-time (Static)

void fun (int x); void
fun (double);

Overriding

Defining a funcⁿ in derived class
with same signature as in
the base class.

Funcⁿ exist in base & derived class

Require in Base Class

(Runtime binding (dynamic))

virtual void fun();

void fun() override;

Q5] Explain internal implementation of a class with virtual C using VTABLE & UPTR.

- When a class has virtual C, compiler generates:
- 1) VTABLE (Virtual-table) - Table storing pointers to virtual C of class
 - 2) UPTR - A hidden pointer in objects that points to Class's vtable

```

eg:
class Base {
public:
    virtual void show() { cout << "Base" << endl; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived" << endl; }
};

int main() {
    Base* bptr = new Derived();
    bptr -> show(); // calls Derived::show() using VTABLE
}

```

Q4] What is Upcasting & Downcasting?

→ 1) Upcasting : → Pointer having less capacity, points to large data capacity pointer.

2) Downcasting : Pointer points to data having less capacity than pointer who is pointing.

→ if we use downcasting → will get compiler error

Code :: *-virtual2.cpp

→

Q5] If base class & derived class contains same named & prototype method then which is considered as:

- 1) overloading - Same funcⁿ name but diff parameters
- 2) overriding - Same —, same parameters but defined in base & derived classes

2) Redefinition: Same function name & parameters in a new scope (w/o virtual)

1) In which scenario of class first 4 bytes are reserved as VPTR in object layout?

2) When a class contains at least 1 virtual(), VPTR is added as 1st 4 bytes in object layout.

3) These bytes store the address of VTABLE.

3) Necessary things for runtime polymorphism:

1) Base Class pointer (reference to derived class object).

2) virtual() in Base Class.

3) Function overriding in derived class.

4) Dynamic memory allocation using 'new'.

4) Can we override a private virtual() from Base Class?

1) Yes, but it must be accessible in derived class.

- A private virtual() can be overridden, but it can't be called from derived class.

5) Class Base {

private:

virtual void show() { cout << "Base" << endl; }

protected:

virtual void display() { cout << "Base display" << endl; }

};

Class Derived: public Base {

public:

void display() override { cout << "Derived" << endl; }

};

int main() {

Derived d;

d.display(); // works

// d.show(); // Error: show() is private }

- Q1] Why concept of Downcasting is not allowed
→ - It breaks type safety because :
- 1) A base class pointer might not actually refer to a derived object.
 - 2) It can lead to undefined behaviour.
 - 3) Leads to - crashes, memory corruption, undefined behaviour.

Q2] Can we define a Static Virtual C() in a Class?
→ No, we cannot have a static virtual C() in C++.

A virtual C() enables runtime polymorphism, while a static func. belongs to Class itself rather than an object.

Why Not :

- 1) Virtual C() need an object for dynamic binding.
- 2) Static functions are not associated with objects, so they can't be overridden dynamically.

Invalid Code :

```
Class Base {
```

```
public:
```

```
    virtual static void showC(); // Error: virtual C() can't be static  
};
```